PROBLEMA:	**OAT 3 - Indexando os meus dados**	**DATA:**	**29/09/2026**
PILOTO (A):		**COPILOTO (A):**	
QA:		**ARQUITETO (A):**	
SCRUM MASTER:			

Objetivo: Compreender a transição das buscas lineares para o conceito de indexação chave-valor e implementar a estrutura genérica fundamental (Struct) que suportará nossa árvore.

Instrução: O Scrum Master tem 10 minutos para guiar o debate. O Arquiteto registra os conceitos diretamente abaixo. A equipe tem 40 minutos para o código.

Fatos:

1. **Regra da Chave de Indexação:** Na OAT 3, qual atributo será utilizado como o critério (chave) para decidir se a navegação na árvore vai para a esquerda ou para a direita?
    - **Fato Extraído:**
2. **Desacoplamento Genérico:** A árvore não pode ser atrelada a uma única classe (ex: apenas Clientes). Qual é a recomendação arquitetural da OAT para armazenar a referência do dado dentro do Nó?
    - **Fato Extraído:**

Brainstorm Investigativo:

1. **O Desafio da Estrutura:** Inspirados no conceito de chave-valor das Tabelas Hash e Mapas da aula de hoje, precisamos criar um Nó de Árvore que guarde a chave

(ID) e o valor (Objeto). O que acontece se o objeto genérico referenciado for nulo na hora da indexação?

- **Decisão da Equipe:**

Cenários de Teste:

- **ID do Teste:** QA-01 - CRIAÇÃO DE NÓ GENÉRICO
- **Responsável (QA):** [Nome] | **Data/Hora:** [DD/MM/AAAA - HH:MM]
- **Contexto:** Validação da estrutura No.java.
- **Cenário:** Dado que INICIALIZAMOS UM NOVO NÓ COM ID 10 E UM OBJETO 'CLIENTE'. Quando VERIFICARMOS AS PROPRIEDADES DO NÓ. Então A REFERÊNCIA DE 'ESQUERDA' E 'DIREITA' DEVEM SER NULAS E A ALTURA DEVE SER [Preencher valor inicial].
- **Status:** [⏳ Pendente | ✅ Passou | ❌ Falhou] | **Observações:** [Justificativa se falhar]

Entrega do Encontro:

O Piloto deve criar e versionar o arquivo No.java contendo os atributos id, registro, altura, esquerda e direita.

PROBLEMA:	**OAT 3 - Indexando os meus dados**	**DATA:**	**06/10/2026**
PILOTO (A):		**COPILOTO (A):**	
QA:		**ARQUITETO (A):**	
SCRUM MASTER:			

Objetivo: Implementar a lógica de inserção seguindo a regra das Árvores Binárias de Busca (menores à esquerda, maiores à direita) e provar a eficiência do motor de busca.

Instrução: O Scrum Master controla o tempo (15 min debate, 45 min código). Preencham os dados abaixo de forma objetiva.

Fatos:

1. **Regra de Inserção:** Ao descer pelos nós comparando o ID desejado com a raiz atual, para onde vai um ID menor e para onde vai um ID maior?
    - **Fato Extraído:**
2. **Eficiência Temporal:** Qual é a complexidade assintótica (Notação Big-O) exigida pela diretoria para localizar um Cliente, Peça ou OS específica de forma otimizada?
    - **Fato Extraído:**

Brainstorm Investigativo:

1. **O Desafio da Recursão vs Iteração:** A busca e a inserção descem pela árvore nó a nó. Vocês implementarão métodos estáticos recursivos ou iterativos (com laços while) para essas operações? Qual o impacto na memória RAM para árvores muito profundas?
    - **Decisão da Equipe:**

Cenários de Teste:

- **ID do Teste:** QA-02 - BUSCA OTIMIZADA LOGARÍTMICA
- **Responsável (QA):** [Nome] | **Data/Hora:** [DD/MM/AAAA - HH:MM]
- **Contexto:** Localização de um registro existente através do motor da Árvore.
- **Cenário:** Dado que INSERIMOS OS IDs 50, 30 e 70. Quando BUSCARMOS O ID 30. Então O SISTEMA DEVE DESCER À ESQUERDA DA RAIZ E RETORNAR O OBJETO CORRESPONDENTE EM FRAÇÕES DE SEGUNDO.
- **Status:** [⏳ Pendente | ✅ Passou | ❌ Falhou] | **Observações:** [Justificativa se falhar]

Entrega do Encontro:

Métodos estáticos **inserir(int id, Object registro)** (ainda sem balanceamento) e **buscar(int id)** integrados na classe gerenciadora.

PROBLEMA:	**OAT 3 - Indexando os meus dados**	**DATA:**	**13/10/2026**
PILOTO (A):		**COPILOTO (A):**	
QA:		**ARQUITETO (A):**	
SCRUM MASTER:			

Objetivo: Substituir os motores de ordenação lentos (Bubble Sort) por algoritmos de travessia nativos de árvores para a exibição de relatórios.

Instrução: Debate expresso de 10 minutos. Foco total em garantir a listagem ordenada de todos os tipos de dados do sistema.

Fatos:

1. **Exibição Ordenada:** Qual é o algoritmo específico de percurso em árvore que garante que os dados sejam impressos de forma estritamente ordenada e crescente, baseada no código identificador?
    - **Fato Extraído:**
2. **Remoção (Conceitual):** Embora a OAT não exija a deleção completa de nós da árvore AVL, a aula de hoje discute a complexidade da remoção. Como a equipe vai lidar caso um dado seja deletado do arquivo CSV? Farão exclusão física na árvore ou apenas uma inativação lógica no objeto referenciado?
    - **Fato Extraído:**

Brainstorm Investigativo:

1. **O Desafio do Cast de Dados:** O percurso Em-Ordem vai retornar uma série de **Object**. Como o sistema, no momento de exibir um "Catálogo de Peças", vai saber transformar e formatar esse objeto genérico para extrair o nome e o preço da Peça na tela?

- **Decisão da Equipe:**

Cenários de Teste:

- **ID do Teste:** QA-03 - LISTAGEM EM-ORDEM CRESCENTE
- **Responsável (QA):** [Nome] | **Data/Hora:** [DD/MM/AAAA - HH:MM]
- **Contexto:** Geração de relatórios sem uso de arrays temporários.
- **Cenário:** Dado que INSERIMOS OS IDs FORA DE ORDEM: 10, 5, 20, 2. Quando SOLICITARMOS A EXIBIÇÃO GERAL. Então O ALGORITMO DEVE IMPRIMIR NA TELA EXATAMENTE A SEQUÊNCIA: 2, 5, 10, 20.
- **Status:** [⏳ Pendente | ✅ Passou | ❌ Falhou] | **Observações:** [Justificativa se falhar]

Entrega do Encontro:

Método **exibirEmOrdem(No raiz)** perfeitamente implementado, substituindo listagens baseadas em Array.

PROBLEMA:	**OAT 3 - Indexando os meus dados**	**DATA:**	**20/10/2026**
PILOTO (A):		**COPILOTO (A):**	
QA:		**ARQUITETO (A):**	
SCRUM MASTER:			

Objetivo: Transformar a BST simples construída até agora em uma verdadeira Árvore AVL, garantindo desempenho contínuo através do cálculo de alturas e rotações automáticas.

Instrução: Encontro complexo. Usem 20 minutos para desenhar (no papel ou lousa virtual) as rotações antes de codificar.

Fatos:

1. **Regra de Balanceamento:** Para que a árvore seja considerada AVL, a inserção deve respeitar rigorosamente uma regra. Quais são os únicos valores permitidos para o fator de balanceamento de qualquer nó?
    - **Fato Extraído:**
2. **Correção Automática:** Sempre que um desbalanceamento for detectado, o sistema deve executar ações corretivas. Quais são as quatro rotações possíveis exigidas no projeto?
    - **Fato Extraído:**

Brainstorm Investigativo:

1. **O Desafio da Altura:** Como a propriedade altura do No será atualizada? A atualização deve acontecer na "volta" da recursão após uma inserção? Como calcular o fator de balanceamento (Altura da Esquerda - Altura da Direita)?
    - **Decisão da Equipe:**

Cenários de Teste:

- **ID do Teste:** QA-04 - ROTAÇÃO SIMPLES À DIREITA
- **Responsável (QA):** [Nome] | **Data/Hora:** [DD/MM/AAAA - HH:MM]
- **Contexto:** Validação de desbalanceamento crítico do lado esquerdo.
- **Cenário:** Dado que A RAIZ É 3 E INSERIMOS 2 (FICANDO À ESQUERDA). Quando INSERIRMOS O ID 1. Então A ÁRVORE DEVE DETECTAR DESBALANCEAMENTO (+2) NO NÓ 3 E EXECUTAR UMA ROTAÇÃO, TORNANDO O NÓ 2 A NOVA RAIZ.
- **Status:** [⏳ Pendente | ✅ Passou | ❌ Falhou] | **Observações:** [Justificativa se falhar]

Entrega do Encontro:

Métodos calcularAltura(), fatorBalanceamento() e os 4 métodos de Rotação integrados ao processo de inserção do Indexador Principal na *Branch Main*. Fim da codificação obrigatória para a OAT!